



Scrye User & Plugin Guide

Everything you need to use Scrye, plus a full reference
for writing plugins.

For everyone

Scrye is a cross-platform MUD client (C# / .NET 10, Avalonia UI). It connects to text MUDs, renders their ANSI/MXP output, and gives you the usual automation — triggers, aliases, timers, macros, highlights — plus a profile system, themeable UI, capture panes, the MIP feed, and a Lua/JavaScript **plugin** system that can add commands and live **HUD panels**.

This guide has two parts:

1. **Using Scrye** — for everyone.
 2. **Writing plugins** — for plugin authors, with a full `scrye.*` API reference.
-

Part 1 — Using Scrye

Connecting to a world

A "world" is a MUD connection. You can connect two ways:

- **Quick connect** — type a host and port and go. This is session-only: nothing is saved.
- **Saved profiles** — a world defined by a MUD, and optionally an Account and a Character under it. Saved worlds remember their settings and reconnect the same way each time.

Each connected world gets its own tab with an output pane and a command line.

The main window

- **Output pane** — the MUD's text, with ANSI colors, MXP links, and clickable command links. It has its own scrollback; scroll up to read history, and it snaps back to the bottom on new output.
- **Command line** — type a command and press **Enter** to send it. Handy keys:
- **Up / Down** — walk back and forth through command history.
- **Tab** — complete the word under the caret from words seen in the output.
- **Ctrl+F** — open the find bar to search the scrollback.
- **Esc** — clear the input.

Profiles and the cascade

Settings resolve through a four-layer cascade, from most general to most specific:

Global → MUD → Account → Character

Deeper layers win for single values (theme, font), and collections (triggers, aliases, macros, variables) are merged across layers. This means you can set something once globally and override it for one character. The **Global** layer is edited in **Settings**; MUD/Account/Character layers are edited from the world/profile UI.

Automation

- **Triggers** — match lines of MUD output (plain text or regex) and react: send commands and/or run script. A trigger can send **multiple commands** — separate them so each goes on its own line.
- **Aliases** — match what *you* type and rewrite/act on it. A matching alias consumes your input.
- **Timers** — run on an interval.
- **Sequences** — a named list of commands you can fire together (e.g. a walk).

- **Highlights** — recolor matching text in the output without changing anything else.
- **Macros / keybindings** — bind a key (e.g. `F1`, `Ctrl+K`, `NumPad1`) to send a command or run an action.

These are all editable in Settings (Global) or per world/character.

Appearance

In **Settings** → **Appearance**:

- **Theme** — several dark/light color schemes with different accents. The game **output pane stays a constant near-black** across all themes so the MUD's own colors always read correctly.
- **MUD colors (ANSI palette)** — choose how the MUD's ANSI color codes are painted:
- **Modern (xterm)** — the softer xterm/VGA palette (default).
- **MUSHclient (classic)** — MUSHclient's default palette (pure-primary bright colors, olive yellow), if you want it to look exactly like MUSHclient. Applies to new output.
- **Font** — a dropdown of the **monospaced** fonts installed on your machine, with a live preview. MUD output needs a fixed-width font so columns line up; the picker only lists monospaced fonts so you can't break alignment by accident. There's also an "Advanced" box for a custom comma-separated fallback chain.
- **Font size.**

Scrye also lifts any near-black text the MUD sends so it stays legible on the dark background — bright colors are left untouched.

HUD panels

Plugins can contribute **HUD panels** — small floating widgets (status bars, gauges, maps, buttons) that sit over the output and stay in sync with game state. You can **drag panels** by their title to reposition them so they don't overlap; positions are remembered.

Capture panes

A **capture pane** is a separate scrolling pane that collects specific lines — for example, all chat channels and tells routed into one "Chats" pane. Plugins (and triggers) can route lines into named panes.

Plugins

Plugins add commands and HUD panels. Manage them in the **Plugins** panel for a world:

- Plugins are **opt-in per character** — enabling one for a character doesn't add it to every character.

- Each plugin can be **enabled** / **disabled** / **reloaded** / **removed**.
- **Reload** re-reads the plugin's script from disk, so you can edit a Lua plugin and reload it live — no restart needed (this works for script-only changes; changes to Scrye itself need a rebuild).

Where files live

Scrye stores its data under your user profile:

What	Location
Profiles	%APPDATA%/Scrye/profiles
Plugin storage (<code>scrye.store</code>)	%APPDATA%/Scrye/plugin-data/<world>/<plugin-id>.json
Plugin logs (<code>scrye.log</code>)	%APPDATA%/Scrye/logs/plugins/<plugin-id>.log
Crash / session logs	%APPDATA%/Scrye/logs
User plugins	%APPDATA%/Scrye/plugins (also loaded from the <code>plugins</code> folder next to the app)
Sounds	%APPDATA%/Scrye/sounds

(On macOS/Linux "%APPDATA%" maps to the platform's application-data folder.)

Part 2 — Writing plugins

A plugin is a folder with a manifest and a script. When enabled for a character, its script runs once (registering hooks, timers, rules, and panels), then reacts to game events for the life of the session.

Anatomy

```
my-plugin/  
  plugin.json    ← manifest  
  main.lua       ← entry script
```

Drop the folder into `%APPDATA%/Scrye/plugins/` (or the `plugins` folder next to the app), then enable it from the Plugins panel and hit **Reload** after edits.

The manifest — `plugin.json`

```
{  
  "id": "my-plugin",  
  "name": "My Plugin",  
  "version": "1.0.0",  
  "author": "Your Name",  
  "description": "What it does, in one line.",  
  "mudIds": ["*"],  
  "entry": "main.lua",  
  "lang": "lua",  
  "enabled": true  
}
```

Field	Meaning
<code>id</code>	Unique id. Used for storage, logs, and its state namespace. Keep it stable.
<code>name</code>	Display name in the Plugins panel.
<code>version</code> , <code>author</code> , <code>description</code>	Metadata.
<code>mudIds</code>	Which worlds it applies to. <code>["*"]</code> (or empty) = all worlds; otherwise a list of MUD ids.
<code>entry</code>	Entry script relative to the folder. Default <code>main.lua</code> .
<code>lang</code>	<code>"lua"</code> (MoonSharp) or <code>"js"</code> (Jint). Default <code>lua</code> .
<code>enabled</code>	Whether it's a candidate to load. Users still opt in per character.

The scripting environment

Lua plugins run on **MoonSharp** (a managed Lua 5.2-ish engine) in a **soft sandbox**: you get the standard library — `string`, `table`, `math`, `os.time` / `os.date` / `os.clock`, `pcall`, `tonumber`, etc. — but **no** `io`, **no** `os.execute`, **no** `filesystem`, **no** `require`. For durable data use `scrye.store` (key/value) and `scrye.log` (a log file); the host handles the file I/O for you.

JavaScript plugins run on **Jint** and see the same `scrye.*` API (with JS idioms: arrays instead of Lua tables, `onSubmit` / `onClick` as functions, etc.).

Everything runs on the session's loop thread, so your script never re-enters concurrently — you don't need locks.

The `scrye.*` API

Output & input

Call	Description
<code>scrye.print(text)</code>	Echo a line to the local output, tagged with your plugin id.
<code>scrye.send(text)</code>	Send a command to the MUD.
<code>scrye.log(text)</code>	Append a line to your plugin's log file.

State & variables

The **state tree** is a shared key/value space the whole app reads from. Game feeds publish into it (e.g. `character.health.current`, `enemy.name`, and MUD-specific feeds like `vik.*`). Plugins publish their own derived state under `plugin.<id>.*` and bind HUD widgets to it.

Call	Description
<code>scrye.getState(path)</code>	Read a state value as a string ("" if unset).
<code>scrye.setState(path, value)</code>	Publish a value (use <code>"plugin." .. scrye.id .. ".key"</code>).
<code>scrye.watch(path, function(value, path) ... end)</code>	React whenever a state path (or subtree) changes.
<code>scrye.getVariable(name) /</code> <code>scrye.setVariable(name, value)</code>	Read/write a world variable (shared across plugins & triggers).

Event hooks

Call	Fires when
<code>scrye.onLine(function(line) ... end)</code>	Each output line. Return false to gag the line, or return a string to rewrite it.
<code>scrye.onPrompt(function() ... end)</code>	The MUD shows a prompt.
<code>scrye.onConnect(fn) / scrye.onDisconnect(fn)</code>	Connection opens / closes.
<code>scrye.onGmcp(function(json, package) ... end)</code> or <code>scrye.onGmcp("Char.Vitals", fn)</code>	A GMCP message (optionally filtered by package).
<code>scrye.onChannel(function(channel, message) ... end)</code> or <code>scrye.onChannel("Party", fn)</code>	A MIP chat message. Tells arrive with channel <code>"Tell"</code> .

Timers

Call	Description
<code>scrye.after(seconds, fn) → id</code>	Run once after a delay.
<code>scrye.every(seconds, fn) → id</code>	Run repeatedly.
<code>scrye.cancel(id)</code>	Cancel a timer.

Timer granularity is ~1 second.

Rules

Call	Description
<code>scrye.addTrigger{ pattern=, regex=, ignoreCase=, send=, run= }</code>	Match output; <code>send</code> a command and/or <code>run</code> a function.
<code>scrye.addAlias{ ... }</code>	Match what the user types; a match consumes the input.

`pattern` is required. `regex=true` for regex (else plain substring). `ignoreCase` defaults to true. `run` receives regex capture groups as arguments.

Persistent storage

Survives restarts (`%APPDATA%/Scrye/plugin-data/<world>/<id>.json`), scoped to your plugin:

```
scrye.store.set("key", "value")
local v = scrye.store.get("key")  -- string or nil
scrye.store.delete("key")
local keys = scrye.store.keys()   -- { "k1", "k2", ... }
```

Values are strings — serialize tables yourself (e.g. join with a delimiter).

Alerts & routing

Call	Description
<code>scrye.notify(text)</code>	Toast notification (+ taskbar flash when unfocused).
<code>scrye.sound("beep")</code>	Play <code>"beep"</code> , a path, or a file in the sounds folder.
<code>scrye.capture(pane, text)</code>	Route a line into a named capture pane.

HUD panels

`scrye.addPanel{ ... }` contributes a declarative panel. The host renders it and keeps bound widgets in sync with state.

```
scrye.addPanel{
  title = "My Panel",
  width = 300,
  accent = "#5A93D4",  -- title + border color (optional)
  background = "#101418", -- panel fill (optional)
  color = "#D6DEE8",   -- default text color for widgets (optional)
  widgets = { ... },   -- OR tabs = { { title=, widgets={...} }, ... }
}
```

Content is either a flat `widgets` list **or** a set of `tabs`.

HUD widget reference

Each widget is a table with a `type`. Common fields: `text` (a label/prefix), `bind` (a state path to display), `color` (a `#RRGGBB` override).

type	What it shows	Key fields
<code>label</code>	Static (or bound) text.	<code>text</code> , or <code>bind</code> ; <code>color</code>
<code>value</code>	A prefix plus a live value.	<code>text</code> (prefix), <code>bind</code> ; <code>color</code>
<code>text</code>	A multi-line monospaced block (reports/tables).	<code>bind</code> ; <code>color</code>
<code>gauge</code>	A labeled bar; auto-colors green→amber→red by ratio unless <code>color</code> set.	<code>text</code> , <code>value</code> , <code>max</code> (state paths or numbers); <code>color</code>
<code>progress</code>	A labeled bar with an explicit color.	<code>text</code> , <code>value</code> , <code>max</code> ; <code>color</code>
<code>button</code>	A clickable button.	<code>text</code> , <code>action = function() ... end</code>
<code>buttonrow</code>	Several buttons side by side (equal width).	<code>buttons = { {text=, action=}, ... }</code>
<code>input</code>	An inline text field; Enter or the Set button submits.	<code>text</code> (label), <code>bind</code> (seed value), <code>onSubmit = function(text) ... end</code>
<code>colorgrid</code>	A clickable grid of characters, colored by a palette.	<code>bind</code> (grid string), <code>palette = { ["#"]="#RRGGBB", ... }</code> , <code>onClick = function(col, row, ch) ... end</code>

Notes:

- **Dynamic content** flows through `bind` + `setState`: the set of widgets is fixed when the panel is built, but their bound text updates live. You can't add or remove widgets at runtime, so for a variable-length list either use a single `text` widget or a `colorgrid`.
- `value` / `max` on gauges/progress accept a **state path** or a **literal number** (e.g. `max = 100`).
- `colorgrid` `onClick` gives you the clicked cell's `col`, `row`, and character — map that back to your data.
- `input` seeds its field from `bind`; refresh it by `setState`-ing that key.

State namespaces you'll see

- `plugin.<id>.*` — your own published state (bind widgets here).
- `character.*` — vitals etc. (e.g. `character.health.current`, `character.health.max`).
- `enemy.name`, `enemy.health` — current target.
- Game-specific feeds — e.g. on 3Sapes the viking MIP feed lands under `vik.*` (`vik.daler`, `vik.wstock`, `vik.carts`, `vik.buildings`, ...), readable by any plugin via `scrye.getState("vik.<key>")` and watchable with `scrye.watch("vik", fn)`.

Installing, reloading, packaging

- **Install:** drop the plugin folder into `%APPDATA%/Scrye/plugins/`, enable it for a character in the Plugins panel.
- **Reload:** after editing the script, click **Reload** — it re-reads from disk live.
- **Share:** zip the plugin folder (the `plugin.json` must be at the archive root or in a single top folder). The recipient unzips it into their plugins folder.

MoonSharp gotchas (worth knowing)

MoonSharp is a managed reimplement of Lua, and a couple of its quirks have bitten real plugins. None are hard to avoid:

- **Initialize your locals.** An *uninitialized* `local x` inside a busy function with nested loops/closures can, in rare cases, come back as a stale value instead of `nil`. Write `local x = nil` (or `= 0`, `= false`) explicitly. Explicitly-initialized locals are always fine.
- **Keep patterns simple / gate them.** A Lua pattern that has to backtrack heavily (e.g. `town + two numbers + keyword` run against a line that doesn't fit) can abort with **"pattern too complex."** Gate an expensive `:match` behind a cheap `:find` of a keyword first, and prefer simple, well-anchored patterns.
- **Timers tick at ~1s.** Don't rely on sub-second `scrye.after`.
- **Colors are #RRGGBB strings.** The host builds thread-safe brushes for you; just pass hex.
- **State is strings.** Numbers come back as text — `tonumber(...)` them.

A complete example

A tiny plugin that watches your HP, warns when it's low, adds an `hp` command, and shows a gauge:

`plugin.json`

```
{
  "id": "hp-watch",
  "name": "HP Watch",
  "version": "1.0.0",
  "author": "You",
  "description": "Low-HP warning + a gauge",
  "mudIds": ["*"],
  "entry": "main.lua",
  "lang": "lua",
  "enabled": true
}
```

`main.lua`

```

local P = "plugin." .. scrye.id .. "."

-- warn once when HP drops below 30%
local warned = false
scrye.watch("character.health.current", function()
    local cur = tonumber(scrye.getState("character.health.current")) or 0
    local max = tonumber(scrye.getState("character.health.max")) or 1
    local pct = max > 0 and (cur * 100 / max) or 100
    scrye.setState(P .. "hpline", string.format("HP %d/%d (%d%%)", cur, max, math.floor(pct)))
    if pct < 30 and not warned then
        warned = true
        scrye.notify("Low HP!")
        scrye.sound("beep")
    elseif pct >= 30 then
        warned = false
    end
end)

-- an alias: typing "hp" prints the current line
scrye.addAlias{
    pattern = "^hp$", regex = true,
    run = function() scrye.print(scrye.getState(P .. "hpline")) end,
}

-- a small HUD panel with a live gauge and the text line
scrye.addPanel{
    title = "HP Watch",
    width = 240,
    accent = "#D6524E",
    widgets = {
        { type = "gauge", text = "HP",
          value = "character.health.current", max = "character.health.max" },
        { type = "value", text = "", bind = P .. "hpline", color = "#E0A830" },
    },
}

```

Reload it, and you get a gauge, an `hp` command, and a low-HP warning — a good template to build from.

Scrye is a clean-room successor to MUSHclient. If you're porting a MUSHclient plugin:
miniwindows become `scrye.addPanel`, `SetVariable` / `GetVariable` config becomes `scrye.store`, `DoAfterSpecial` becomes `scrye.after`, `Send` / `Note` become

`scrye.send` / `scrye.print`, and *inputboxes* become `input` widgets or `atrade`-style commands.